Pràctica 2: Monitorització Ambiental d'un CPD

"Quan els ordinadors prenguin el control, potser no el recuperarem. Sobreviurem segons el seu caprici. Amb sort, decidiran mantenir-nos com a mascotes."
Revista Life, novembre de 1970

Generalitat de Catalunya
Departament d'Educació
Sistemes Microinformàtics i Xarxes
Projectes_DANI S.
Institut de Lliçà

Alex Nicasi i Jonathan Cruz
2n SMX A
12/03/26

Document tècnic

Contingut del document:

1. **Portada**
2. **Índex**
3. **Títol i objectiu de la pràctica**

    - Breu descripció de la pràctica

4. **Explicació de conceptes clau**

    - Què és una classe, un objecte i un mètode.

    - Relació amb l'objecte `Serial`.

5. **Esquema de connexions (fritzing)**
6. **Diagrama de flux del programa (com fer un diagrama de flux)**
7. **proves**
   Enllaç al Vídeo demostratiu del funcionament
8. **Explicació detallada del seguiment del projecte dins de Github**. Amb imatges reals del vostre projecte a GitHub cal explicar tot el procés (repositori, taulell Kanban, etc…)

2) Estructura del repositori GitHub

Cada grup ha de tenir **un repositori** amb aquesta estructura:

```
── practica2/
   ├── README.md        ← Explicació general de la pràctica i objectius
   ├── src/             ← Fitxers de codi (.cpp, .h, .ino…)
   │   └── main.ino
   └── doc/             ← Documentació, captures, PDF, imatges, vídeos
```

3) Configuració de GitHub Project

1. Creeu un **Project** tipus Kanban dins del repositori anomenat `Practica 2 - Monitorització CPD`.

2. Creeu com a mínim les **columnes**:

   🟥 `To do`
   🟨 `Doing`
   🟩 `Done`

Configureu el filtre de la vista per veure només els Issues etiquetats amb practica2.

Flux de treball

1. **To do** → Col·locar totes les tasques planificades (issues).

2. **Doing** → Quan comenceu una tasca, moure-la a Doing (filtrant per label `practica1`).

3. **Done** → Quan la tasca estigui completada, fer **commit** amb missatge clar + `Closes #issue.` per exemple:

```
Afegit document PDF explicatiu del funcionament USB
Closes #5
```

1. Títol i objectiu de la pràctica

L'objectiu d'aquesta pràctica és crear un sistema de monitorització ambiental per a una sala de servidors (CPD). El sistema utilitza un sensor de temperatura LM35 i un sensor de llum LDR per detectar condicions anormals i generar alertes automàtiques. A més, permet consultar l'estat del sistema mitjançant comandes enviades pel Monitor Sèrie.

2. Breu descripció de la pràctica

En aquesta pràctica es treballa amb entrades analògiques de l'ESP32, que permeten llegir valors continus provinents de sensors. El sistema ha de:

- Llegir la lluminositat amb un LDR.
- Llegir la temperatura amb un LM35.
- Activar alertes automàtiques quan es superen llindars.
- Respondre a comandes com **STATUS** i **LED_OFF**.
- Mostrar les dades en temps real pel Monitor Sèrie.

3. Explicació de conceptes clau

Entrades analògiques

A diferència de les entrades digitals (0 o 1), les entrades analògiques permeten llegir valors continus. L'ESP32 utilitza un ADC (Analog-to-Digital Converter) de 12 bits, que converteix un voltatge entre 0V i 3.3V en un valor entre 0 i 4095.

Sensor LDR

Un LDR és una resistència variable que disminueix el seu valor quan rep llum i l'augmenta quan hi ha foscor. Es llegeix mitjançant un divisor de tensió.

Sensor LM35

El LM35 genera 10 mV per cada grau centígrad. Per convertir la lectura analògica a temperatura:

[V = \frac{lectura}{4095} \cdot 3.3] [T = V \cdot 100]

4. Classes, objectes i mètodes. Relació amb Serial

Què és una classe?

Una classe és un model o plantilla que defineix propietats i comportaments.

Què és un objecte?

Un objecte és una instància d'una classe, és a dir, una còpia funcional amb la qual podem interactuar.

Què és un mètode?

Un mètode és una funció que pertany a una classe i defineix accions.

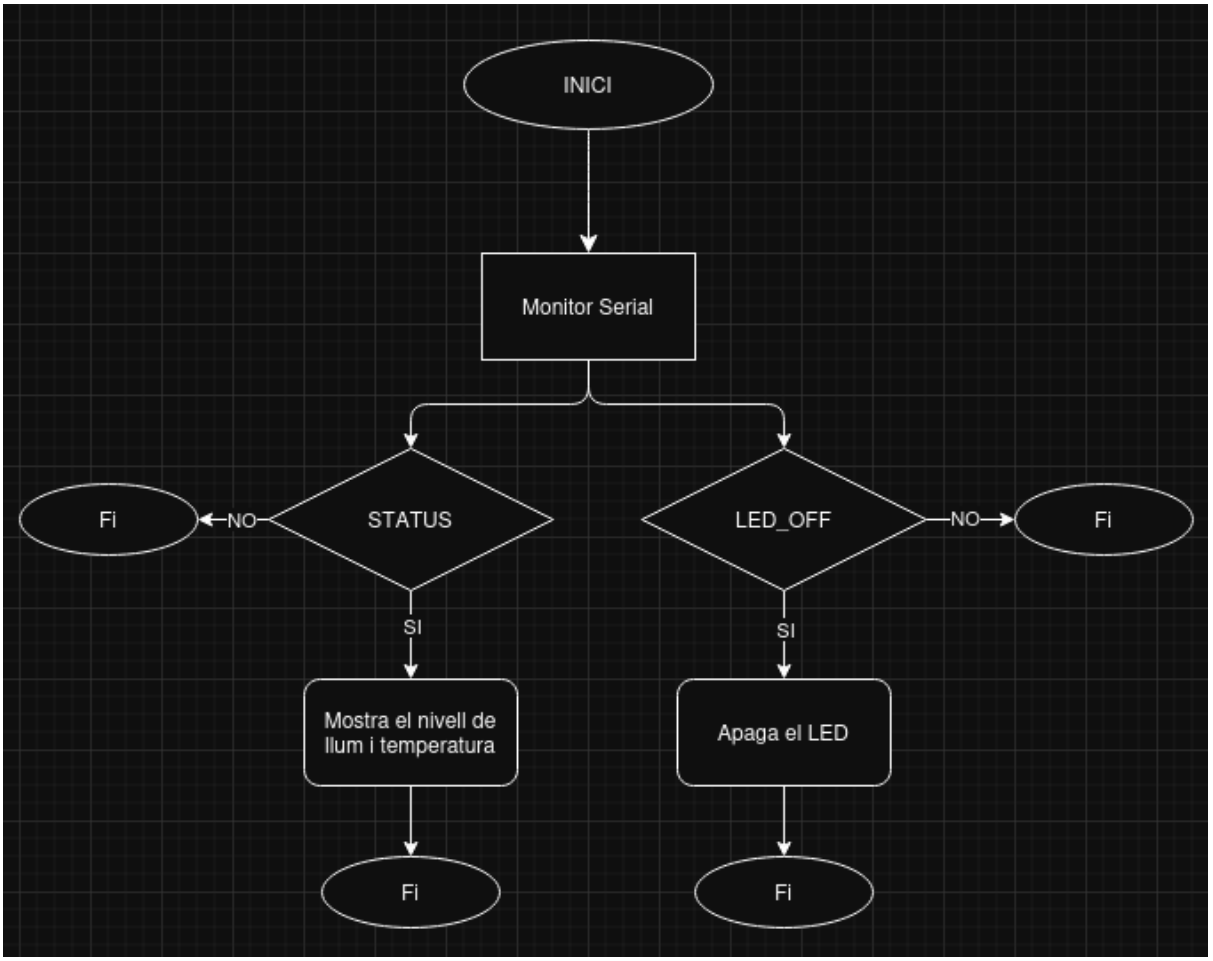
Relació amb l'objecte Serial

`Serial` és un objecte predefinit de la classe **HardwareSerial**. Conté mètodes com:

- `Serial.begin()` → inicialitza la comunicació.
- `Serial.print()` → envia text.
- `Serial.readStringUntil()` → llegeix dades.

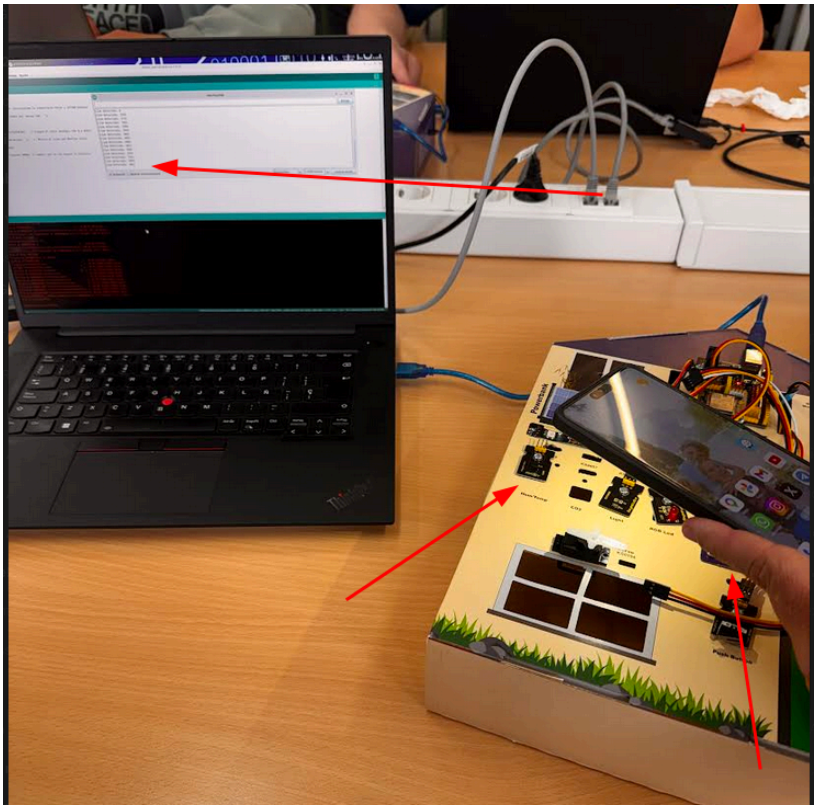
Aquesta pràctica utilitza intensament aquests mètodes per comunicar-se amb l'usuari.

5. Diagrama de flux del programa



6. proves

Issue 1 Lectura del Sensor LDR

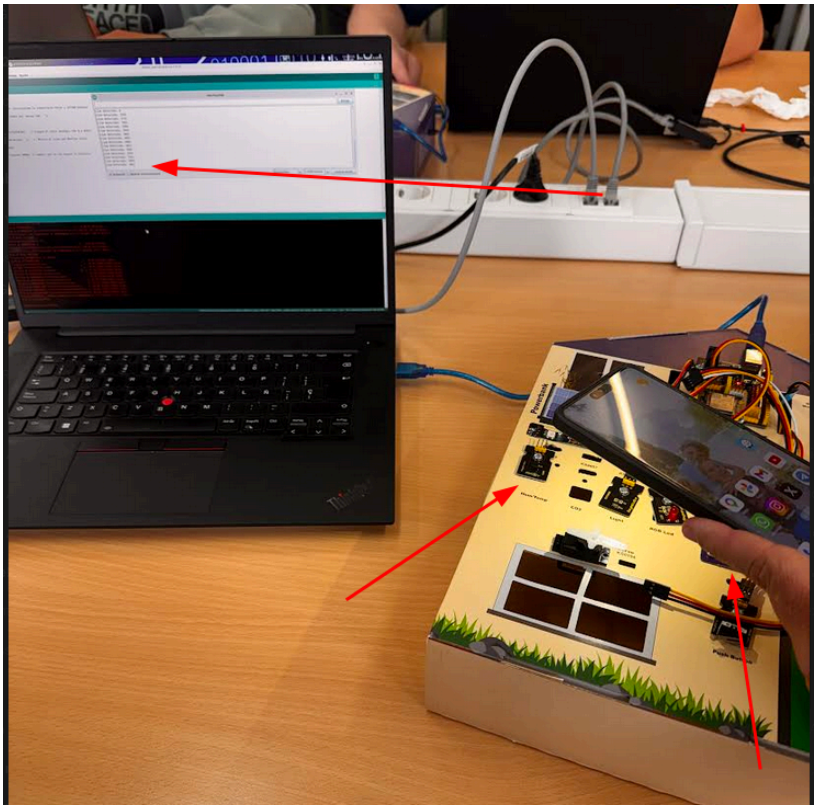


```
const int pinLDR = 2;

void setup() {
  Serial.begin(115200);   // Inicialitzem la comunicació sèrie a 115200 baudios
  delay(1000);
  Serial.println("Llegint dades del sensor LDR...");
}

void loop() {
   int valor = analogRead(pinLDR);   // Llegim el valor analògic (de 0 a 4095)

  Serial.print("Llum detectada: ");   // Mostra el valor pel Monitor Sèrie

  Serial.println(valor);

  delay(2000);  // Esperem 2000ms (2 segons) per no col·lapsar la pantalla

}
```

```
/dev/ttyUSB0

Llum detectada: 1023
Llum detectada: 2077
Llum detectada: 30
Llum detectada: 48
Llum detectada: 0
Llum detectada: 1556
Llum detectada: 1778
Llum detectada: 1802
Llum detectada: 1984
Llum detectada: 3965
Llum detectada: 4095
Llum detectada: 4095
Llum detectada: 4095
Llum detectada: 2015
Llum detectada: 1907

☑ Autoscroll ☐ Mostrar marca temporal
```

7. Explicació detallada del seguiment del projecte dins de Github

Issue 1



```
const int pinLDR = 2;

void setup() {
  Serial.begin(115200);   // Inicialitzem la comunicació sèrie a 115200 baudios
  delay(1000);
  Serial.println("Llegint dades del sensor LDR...");
}

void loop() {
   int valor = analogRead(pinLDR);   // Llegim el valor analògic (de 0 a 4095)

  Serial.print("Llum detectada: ");   // Mostra el valor pel Monitor Sèrie

  Serial.println(valor);

  delay(2000);  // Esperem 2000ms (2 segons) per no col·lapsar la pantalla

}
```

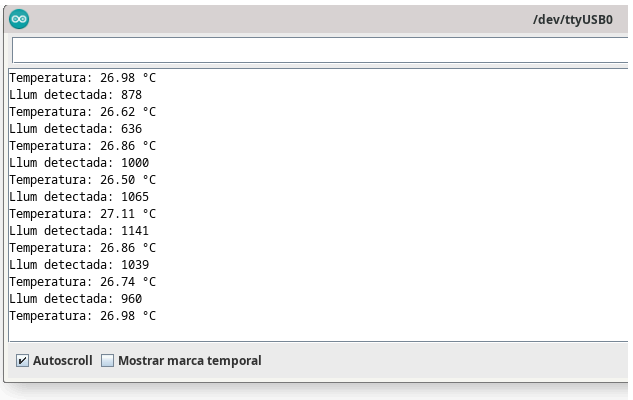
/dev/ttyUSB0

```
Llum detectada: 1023
Llum detectada: 2077
Llum detectada: 30
Llum detectada: 48
Llum detectada: 0
Llum detectada: 1556
Llum detectada: 1778
Llum detectada: 1802
Llum detectada: 1984
Llum detectada: 3965
Llum detectada: 4095
Llum detectada: 4095
Llum detectada: 4095
Llum detectada: 2015
Llum detectada: 1907
```

☑ Autoscroll ☐ Mostrar marca temporal

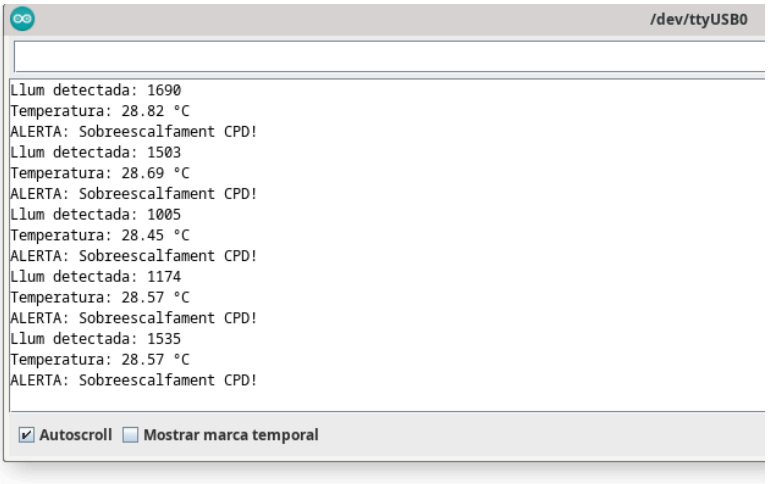
Issue 2

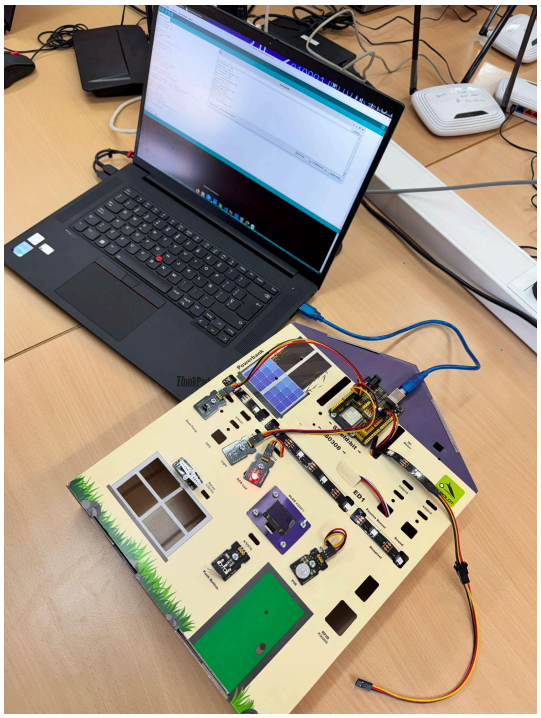
```
const int pinLDR = 2;
const int pinTemp = 4;

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("Llegint sensors...");
}

void loop() {
  // LECTURA LDR
  int valorLDR = analogRead(pinLDR);
  Serial.print("Llum detectada: ");
  Serial.println(valorLDR);

  // LECTURA TEMPERATURA LM35
  int lecturaTemp = analogRead(pinTemp);

  float volts = (lecturaTemp * 5.0) / 4095.0;   // Conversió a volts
  float temperatura = volts * 100.0;            // Conversió a °C

  Serial.print("Temperatura: ");
  Serial.print(temperatura);
  Serial.println(" °C");

  delay(2000);
}
```



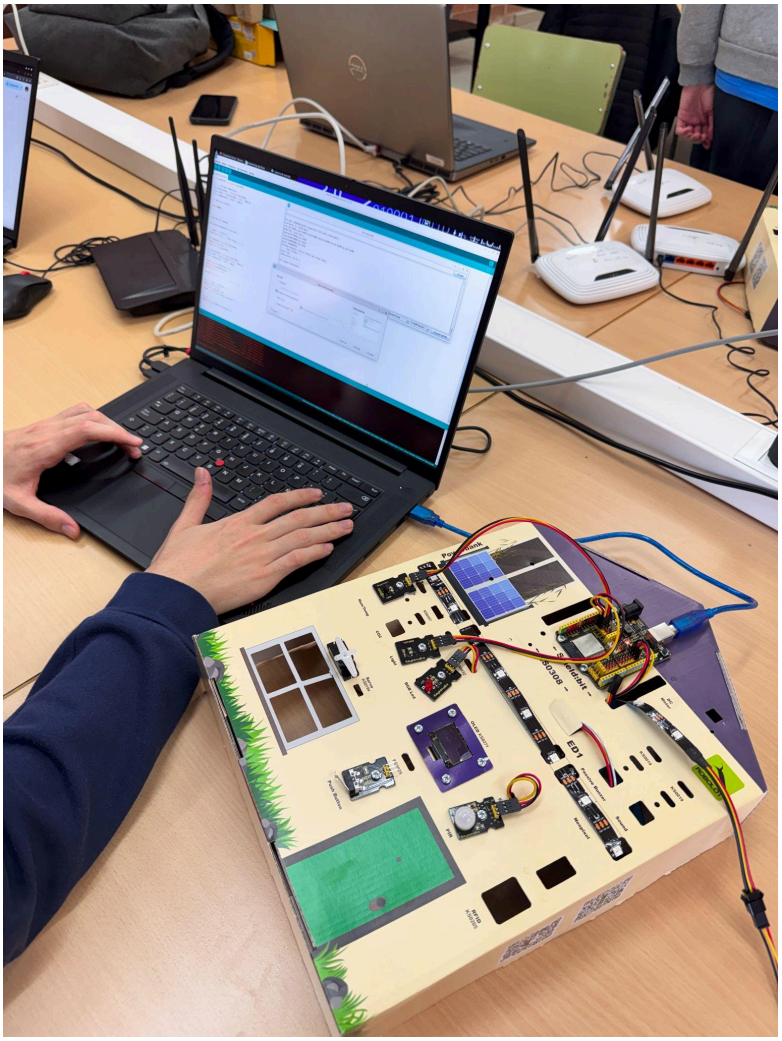
Issue 3

```
const int pinLDR = 2;
const int pinTemp = 4;
const int pinLED = 18;

void setup() {
Serial.begin(115200);
delay(1000);
Serial.println("Llegint sensors...");

pinMode(pinLED, OUTPUT);
digitalWrite(pinLED, LOW);
}

void loop() {
// LECTURA LDR
int valorLDR = analogRead(pinLDR);
Serial.print("Llum detectada: ");
Serial.println(valorLDR);

// LECTURA TEMPERATURA LM35
int lecturaTemp = analogRead(pinTemp);

float volts = (lecturaTemp * 5.0) / 4095.0; // Conversió a volts
float temperatura = volts * 100.0; // Conversió a °C

Serial.print("Temperatura: ");
Serial.print(temperatura);
Serial.println(" °C");

// ALERTA TEMPERATURA
if (temperatura > 25.0) {
digitalWrite(pinLED, HIGH);
Serial.println("ALERTA: Sobreescalfament CPD!");
} else {
digitalWrite(pinLED, LOW);
}

// ALERTA LLUM
if (valorLDR > 2000) {
Serial.println("AVÍS: Porta oberta o llum encesa");
}

delay(2000);
}
```





Issue 4 (main)

```
  int valorLDR = analogRead(pinLDR);

  // --- LECTURA TEMPERATURA ---
  int lecturaTemp = analogRead(pinTemp);
  float volts = (lecturaTemp * 5.0) / 4095.0;
  float temperatura = volts * 100.0;

  // (NO HI HA ALERTES)

  delay(2000);
}

// ----------------------------
// FUNCIONS DE COMANDES
// ----------------------------
void processarComanda(String cmd) {

  if (cmd == "STATUS") {
  Serial.println("---- ESTAT ACTUAL ----");

  int valorLDR = analogRead(pinLDR);
  Serial.print("Llum: ");
  Serial.println(valorLDR);

  int lecturaTemp = analogRead(pinTemp);
  float volts = (lecturaTemp * 5.0) / 4095.0;
  float temperatura = volts * 100.0;

  Serial.print("Temperatura: ");
  Serial.print(temperatura);
  Serial.println(" °C");

  Serial.println("----------------------");
  }

  else if (cmd == "LED_OFF") {
  ledForcatOff = true;
  digitalWrite(pinLED, LOW);
  Serial.println("LED apagat manualment.");
  }

  else {
  Serial.println("Ordre desconeguda.");
  }
}
```

```
/dev/ttyUSB0

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4980
load:0x40078000,len:16612
load:0x40080400,len:3480
entry 0x400805b4
Sistema preparat. Escriu STATUS per veure dades.
---- ESTAT ACTUAL ----
Llum: 1391
Temperatura: 24.79 °C
----------------------
LED apagat manualment.

☑ Autoscroll ☐ Mostrar marca temporal
```